Integrated Mini-SOC Ecosystem

Digital Egypt Pioneers Initiative (Round - 4)

Group : CAI4_ISS6_G1

Track: Infrastructure & Security (Information Security Analyst)

Team Members

Adham Hany Mohamed

Ahmed Mokhtar Helmy

Ibrahim Hussien Ibrahim

Zeinab Ahmed Abdelhamid

1. Project Overview & Objectives

The Mini SOC (Security Operations Center) system is a lightweight, yet fully functional security monitoring environment designed to collect, process, correlate, analyze, and visualize security events from multiple heterogeneous data sources in real-time. It simulates a production-grade SOC infrastructure capable of detecting threats, triaging incidents, generating actionable alerts, and supporting forensic investigation workflows using centralized logging and SIEM capabilities.
This project was developed as the capstone deliverable of the Digital Egypt Pioneers Initiative (DEPI) — Round 4, by a team of four SOC analysts. It bridges the gap between theoretical cybersecurity training and hands-on, real-world defensive operations by building an end-to-end detection and response pipeline from scratch.

1.1 Background & Motivation

Modern organizations face an increasingly sophisticated threat landscape where cyberattacks have evolved from simple opportunistic intrusions to complex, multi-stage operations conducted by organized threat actors. The volume and velocity of security events generated by enterprise environments have grown to a scale that makes manual monitoring not only impractical but operationally impossible.
A Security Operations Center (SOC) provides the centralized capability to continuously monitor, detect, analyze, and respond to cybersecurity threats. This project replicates a scaled-down but architecturally authentic SOC environment using open-source and enterprise-grade tools, enabling the team to develop practical competencies in blue team operations, threat detection engineering, and incident response.

1.2 Project Scope

The scope of this project encompasses the full lifecycle of a SOC deployment, from infrastructure provisioning and agent installation to alert creation, incident triage, and documentation. The environment integrates cloud and on-premises components to reflect realistic enterprise network topologies.
The project covers the following core domains:

- Infrastructure Deployment: Cloud VPS provisioning on Contaboon-premises Windows Server 2022 Domain Controller, Windows 11 Pro workstation, and Ubuntu desktop endpoints.

- SIEM Implementation: Full ELK Stack deployment (Elasticsearch, Logstash, Kibana) with Fleet Server and Elastic Agent for centralized log ingestion from Windows Event Logs, Sysmon, Linux Audited, and network sources.
- Endpoint Detection & Response (EDR): Elastic Defend integration for kernel-level endpoint visibility, process monitoring, file integrity, and behavioral detection across all managed endpoints.
- Identity & Active Directory Security: Monitoring of authentication events, privilege escalation attempts, Kerberoasting indicators, and Group Policy changes via Windows Security Event Logs forwarded from the Domain Controller.
- Threat Detection Engineering: Development of custom detection rules and KQL queries mapped to the MITRE ATT&CK framework to identify adversary techniques including brute force, lateral movement, persistence, and data exfiltration.

1.3 Core Objectives

The project is structured around five strategic objectives that guide every technical and operational decision made during the build and operation of this Mini SOC:

1. **Centralized Visibility:** Aggregate security telemetry from Windows, Linux, and network sources into a single pane of glass, eliminating blind spots across the monitored environment.
2. **Threat Detection & Alerting:** Build and tune detection rules covering the most critical MITRE ATT&CK tactics — from Initial Access and Execution through Persistence, Lateral Movement, and Exfiltration — with measurable low false-positive rates.
3. **Incident Triage Capability:** Establish a structured Tier 1 and Tier 2 triage workflow that enables the analyst team to investigate, classify, and escalate security incidents in alignment with SOC SLA requirements.
4. **Real-World Lab Simulation:** Simulate real attack scenarios (brute force, privilege escalation, lateral movement) within a controlled lab to validate detection coverage and refine response procedures without affecting production systems.

1.4 Technology Stack

The Mini SOC is built on a carefully selected stack of industry-standard and open-source tools that collectively deliver end-to-end security monitoring capabilities:

- **SIEM & Log Management:** ELK Stack 8.x (Elasticsearch, Logstash, Kibana) deployed via Docker on Contabo VPS, providing log storage, processing pipelines, and interactive dashboards.
- **Agent Management & EDR:** Elastic Agent and Elastic Defend (EDR) managed via Fleet Server, deployed on all Windows and Linux endpoints for telemetry collection and behavioral prevention.

- **Identity Infrastructure:** Windows Server 2022 Active Directory Domain Controller (soc.local domain) with Windows 11 Pro domain-joined workstation as a managed endpoint.
- **Detection Framework:** MITRE ATT&CK framework used as the primary reference for detection rule development and threat modeling across all monitored attack surfaces.

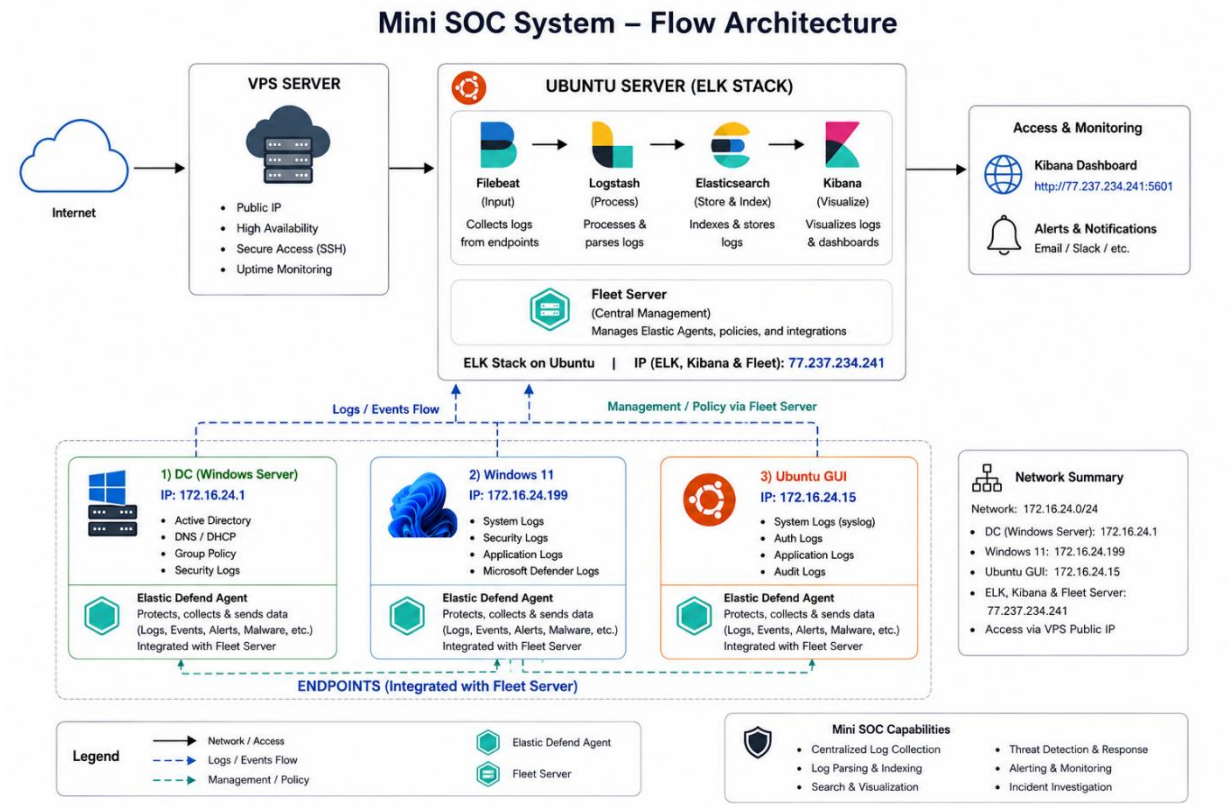
1.5 Expected Outcomes

Upon completion of this project, the team will have delivered the following measurable outcomes:

- A fully operational Mini SOC environment with 24/7 log ingestion from at least four distinct data sources (Windows DC, Windows workstation, Ubuntu endpoint, and network telemetry).
- A library of validated detection rules covering the top MITRE ATT&CK techniques relevant to the monitored environment, with documented false-positive tuning notes.
- Custom Kibana dashboards provide real-time security event visibility, alert status tracking, endpoint health monitoring, and authentication anomaly analysis.
- Documented incident response playbooks aligned to the most common alert types generated by the system, enabling consistent and repeatable analyst workflows.
- A comprehensive technical project report demonstrating each team member's contribution, architectural decisions, challenges encountered, and lessons learned throughout the SOC build process.

2. System Architecture & Lab Design

The infrastructure is distributed across three primary environments to ensure high availability and realistic network segmentation.



A. Contabo VPS — Cloud SOC Platform

- SIEM Stack: Dockerized deployment of ELK Stack (Elasticsearch, Logstash, Kibana).
- Central Management: Fleet Server for managing security policies across all agents which integrated with elastic defend.

B. Adham's Laptop — Infrastructure Cluster

- Identity Management: Windows Server 2022 acting as the Domain Controller (DC).
- Workstation: Windows 11 Pro instance joined to the soc.local domain and Ubuntu GUI Machine to get Linux logs
- Security Layers: Both machines protected by Elastic Agent and Elastic Defend (EDR) for kernel-level visibility.

4. Backend Infrastructure: Elasticsearch & Kibana Deployment

The core of our SOC resides on a Contabo VPS (Ubuntu 22.04) to ensure 24/7 log availability. Elasticsearch 8.x is deployed as the primary search and analytics engine.

Phase 1: Installing Elasticsearch

To ensure stable and secure deployment, follow these commands in order. This process adds the official Elastic repository and installs the latest 8.x version.

Step 1 — Add the GPG Key

```
wget -qO - https://artifacts.elastic.co/GPG-KEY-elasticsearch | sudo gpg --dearmor -o /usr/share/keyrings/elasticsearch-keyring.gpg
```

Step 2 — Add the Official Repository

```
echo "deb [signed-by=/usr/share/keyrings/elasticsearch-keyring.gpg] https://artifacts.elastic.co/packages/8.x/apt stable main" | sudo tee /etc/apt/sources.list.d/elastic-8.x.list
```

Step 3 — Update and Install

```
sudo apt update && sudo apt install elasticsearch -y
```

Phase 2: Performance Optimization (JVM Tuning)

Since we are working with a 12GB RAM environment, we must limit the memory consumption of Elasticsearch to avoid system freezing. We allocate 4GB to the service.

Step 1 — Create Memory Configuration File

```
sudo nano /etc/elasticsearch/jvm.options.d/memory.options
```

Step 2 — Set Heap Size

Inside the file, paste the following lines (ensure there are no leading spaces):

```
-Xms4g
-Xmx4g
```

Press Ctrl+O then Enter to save, and Ctrl+X to exit.

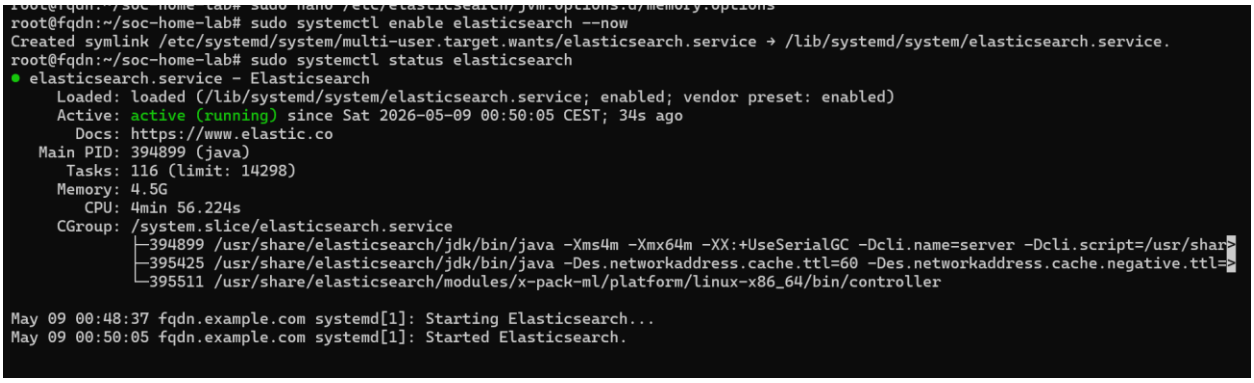
Phase 3: Service Activation & Security

Step 1 — Enable and Start the Service

```
sudo systemctl enable elasticsearch --now
```

Step 2 — Verify Service Status

```
sudo systemctl status elasticsearch
```



Step 3 — Reset Admin Password

By default, security is enabled in version 8.x. You must generate a new password for the elastic superuser:

```
sudo /usr/share/elasticsearch/bin/elasticsearch-reset-password -u elastic
```

Important: Copy the generated password immediately and save it in a secure location.

Step 4 — Connection Test

Verify that the API is responding correctly:

```
curl -u elastic -k https://localhost:9200
```

If successful, you will see a JSON response containing the cluster name and version number.

5. Installing and Configuring Kibana

Since the Elastic repositories were added in the previous phase, we can proceed directly to the installation and integration of Kibana as the visualization interface for our SOC.

Step 1 — Install Kibana

```
sudo apt install kibana -y
```

```
root@fqdn:~/soc-home-lab# sudo apt install kibana -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  kibana
0 upgraded, 1 newly installed, 0 to remove and 5 not upgraded.
Need to get 394 MB of archives.
After this operation, 1221 MB of additional disk space will be used.
Get:1 https://artifacts.elastic.co/packages/8.x/apt stable/main amd64 kibana amd64 8.19.15 [394 MB]
Ign:1 https://artifacts.elastic.co/packages/8.x/apt stable/main amd64 kibana amd64 8.19.15
Get:1 https://artifacts.elastic.co/packages/8.x/apt stable/main amd64 kibana amd64 8.19.15 [394 MB]
Fetched 394 MB in 49s (8116 kB/s)
[master 809cbbd] saving uncommitted changes in /etc prior to apt run
 4 files changed, 6 insertions(+), 2 deletions(-)
 create mode 100644 elasticsearch/jvm.options.d/memory.options
 create mode 120000 systemd/system/multi-user.target.wants/elasticsearch.service
Selecting previously unselected package kibana.
(Reading database ... 162347 files and directories currently installed.)
Preparing to unpack .../kibana_8.19.15_amd64.deb ...
Unpacking kibana (8.19.15) ...
Setting up kibana (8.19.15) ...
Creating kibana group... OK
Creating kibana user... OK
Kibana is currently running with legacy OpenSSL providers enabled! For details and instructions on how to disable see https://www.ela
stic.co/guide/en/kibana/8.19/production.html#openssl-legacy-provider
Created Kibana keystore in /etc/kibana/kibana.keystore
[master 0933327] committing changes in /etc made by "apt install kibana -y"
 13 files changed, 224 insertions(+)
 create mode 100644 default/kibana
 create mode 100644 kibana/kibana.keystore
```

Step 2 — Generate Enrollment Token

To securely link Kibana with Elasticsearch, generate a one-time Enrollment Token:

```
sudo /usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s
kibana
```

Action: Copy the long alphanumeric string generated by this command. You will need it for the initial web configuration.

```
root@fqdn:~/soc-home-lab#     sudo /usr/share/elasticsearch/bin/elasticsearch-create-enrollment-token -s kibana
eyJ2ZXIiOiI4LjE0LjAiLCJhZHIiOlsiNzcuMjM3LjIzNC4yNDE6OTIwMCJdLCJmZ3IiOiIxNmM5M2JmMDdjNTg1OTQzMWQ3ZWEyNzQ4NDA0Y2ZmYjAyZjY1YjA1OWNiNWZlZ
WIwMWM0Y2NlMDZjMzlmMWZjIiwia2V5Ijoid203Y0NaNEJXUjZSeUxLRGEzMHM6SHQzT2dNMmplTGdHaTh3TVVSWm5QdyJ9
root@fqdn:~/soc-home-lab#
```

Step 3 — Enable and Start the Kibana Service

```
sudo systemctl enable kibana --now
```

```
root@fqdn:~/soc-home-lab#     sudo systemctl enable kibana --now
Created symlink /etc/systemd/system/multi-user.target.wants/kibana.service → /lib/systemd/system/kibana.service.
root@fqdn:~/soc-home-lab#
```

Step 4 — Verify Service Status

```
sudo systemctl status kibana
```

Step 5 — Access the Web Interface

Open your browser and navigate to your VPS IP address on port 5601:

```
https://77.237.234.241:5601
```

6. SSL/TLS Certificate Generation

To ensure secure, encrypted communication between Elastic Agents and the Fleet Server, we use the elasticsearch-certutil tool to generate self-signed certificates.

Step 1 — Generate the Certificate Authority (CA)

Create a Certificate Authority to sign certificates for our server nodes:

```
sudo /usr/share/elasticsearch/bin/elasticsearch-certutil ca
```

```
root@fqdn:~/soc-home-lab# sudo /usr/share/elasticsearch/bin/elasticsearch-certutil ca --pem
This tool assists you in the generation of X.509 certificates and certificate
signing requests for use with SSL/TLS in the Elastic stack.

The 'ca' mode generates a new 'certificate authority'
This will create a new X.509 certificate and private key that can be used
to sign certificate when running in 'cert' mode.

Use the 'ca-dn' option if you wish to configure the 'distinguished name'
of the certificate authority

By default the 'ca' mode produces a single PKCS#12 output file which holds:
    * The CA certificate
    * The CA's private key

If you elect to generate PEM format certificates (the -pem option), then the output will
be a zip file containing individual files for the CA certificate and private key
```

```
root@fqdn:~/soc-home-lab# sudo unzip elastic-stack-ca.zip
unzip:  cannot find or open elastic-stack-ca.zip, elastic-stack-ca.zip.zip or elastic-stack-ca.zip.ZIP.
root@fqdn:~/soc-home-lab# sudo /usr/share/elasticsearch/bin/elasticsearch-certutil cert --name fleet-server --ca-cert ca/ca.crt --ca-
key ca/ca.key --pem --dns 77.237.234.241 --ip 77.237.234.241
This tool assists you in the generation of X.509 certificates and certificate
signing requests for use with SSL/TLS in the Elastic stack.

The 'cert' mode generates X.509 certificate and private keys.
    * By default, this generates a single certificate and key for use
       on a single instance.
    * The '-multiple' option will prompt you to enter details for multiple
       instances and will generate a certificate and key for each one
    * The '-in' option allows for the certificate generation to be automated by describing
       the details of each instance in a YAML file

    * An instance is any piece of the Elastic Stack that requires an SSL certificate.
      Depending on your configuration, Elasticsearch, Logstash, Kibana, and Beats
      may all require a certificate and private key.
    * The minimum required value for each instance is a name. This can simply be the
      hostname, which will be used as the Common Name of the certificate. A full
      distinguished name may also be used.
    * A filename value may be required for each instance. This is necessary when the
      name would result in an invalid file or directory name. The name provided here
      is used as the directory name (within the zip) and the prefix for the key and
      certificate files. The filename is required if you are prompted and the name
      is not displayed in the prompt.
```

```
root@fqdn:~/soc-home-lab# sudo mkdir -p /etc/kibana/certs
root@fqdn:~/soc-home-lab# sudo cp fleet-server.crt fleet-server.key ca/ca.crt /etc/kibana/certs/
cp: cannot stat 'fleet-server.crt': No such file or directory
cp: cannot stat 'fleet-server.key': No such file or directory
cp: cannot stat 'ca/ca.crt': No such file or directory
root@fqdn:~/soc-home-lab#
```

7. Troubleshooting & Final Enrollment

Issue: Access Denied Exception

During the deployment phase, the Elasticsearch service failed to initialize due to a critical "Access Denied" error. The service could not read the required security tokens.

Root Cause

Incorrect file permissions and ownership on the service tokens file.

Resolution

- Ownership Adjustment: Transferred ownership of the service tokens file to the elasticsearch user and group.
- Permission Hardening: Set file permissions to 660 — only the service and authorized group can read or write.

Correction Commands

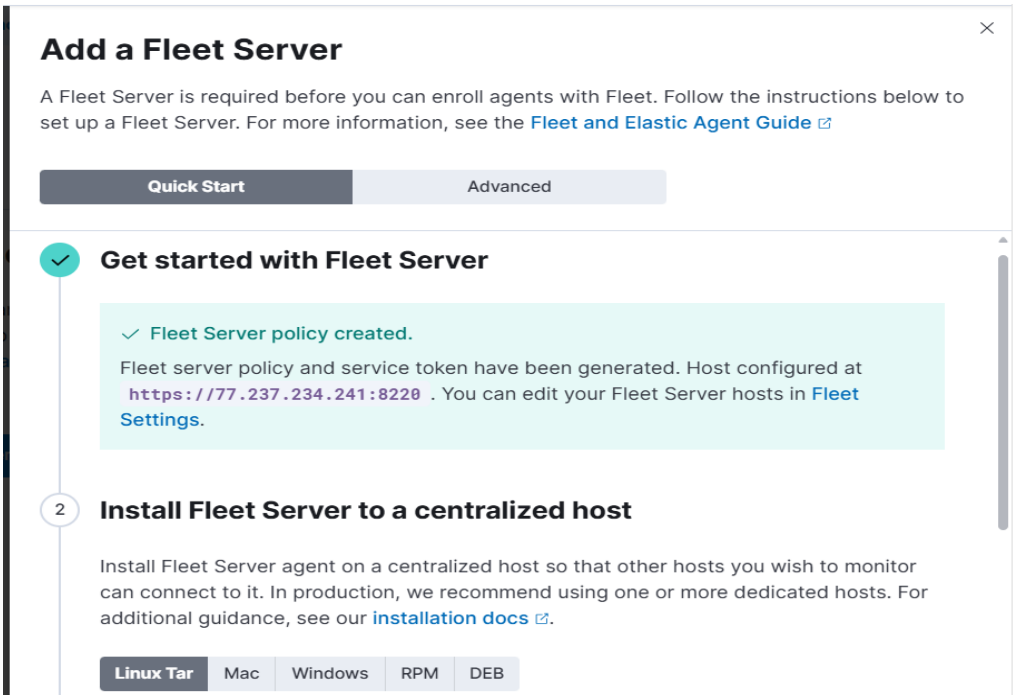
```
# Correcting ownership of the service tokens
sudo chown elasticsearch:elasticsearch /etc/elasticsearch/service_tokens

# Setting secure permissions (Read/Write for Owner & Group)
sudo chmod 660 /etc/elasticsearch/service_tokens
```

Final Fleet Server Enrollment

After confirming that the Elasticsearch service was Active (running) and verifying sufficient system resources (approximately 9.9GB of available memory), we proceeded with the final enrollment of the Fleet Server. To accommodate potential network latency, the operation timeout was increased to 10 minutes.

Final Enrollment Command

```
sudo elastic-agent enroll \
  --url=https://77.237.234.241:8220 \
  --fleet-server-es=https://localhost:9200 \
  --fleet-server-es-ca-trusted-fingerprint=[YOUR_FINGERPRINT] \
  --fleet-server-service-token=[YOUR_TOKEN] \
  --fleet-server-policy=fleet-server-policy \
  --fleet-server-es-insecure \
  --insecure \
  --fleet-server-timeout=10m
```




```
root@fqdn:~/soc-home-lab# curl -L -O https://artifacts.elastic.co/downloads/beats/elastic-agent/elastic-agent-8.11.0-linux-x86_64.ta
.gz
tar xzvf elastic-agent-8.11.0-linux-x86_64.tar.gz
cd elastic-agent-8.11.0-linux-x86_64
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  545M  100  545M    0     0  5752k      0  0:01:37  0:01:37 --:--:-- 6181k
elastic-agent-8.11.0-linux-x86_64/.build_hash.txt
elastic-agent-8.11.0-linux-x86_64/NOTICE.txt
elastic-agent-8.11.0-linux-x86_64/README.md
elastic-agent-8.11.0-linux-x86_64/data/elastic-agent-3c8be7/components/
elastic-agent-8.11.0-linux-x86_64/data/elastic-agent-3c8be7/components/.build_hash.txt
```

```
root@fqdn:~# sudo elastic-agent enroll \
  --url=https://77.237.234.241:8220 \
  --fleet-server-es=https://localhost:9200 \
  --fleet-server-es-ca-trusted-fingerprint=B3AF7D86DB7224768994CDD1FFE0A5627ADDD09DA72595AAED9FA0ED4FB2A487 \
  --fleet-server-service-token=AAEAAWVsYXN0aWMvZmxlZXQtc2VydmVyL3Rva2VuLTE3NzgyODgyMjc0MTM6VXZFb0tta0dRTXk2c0hRdXhNNW1qZw \
  --fleet-server-policy=fleet-server-policy \
  --fleet-server-es-insecure \
  --insecure \
  --fleet-server-timeout=15m
This will replace your current settings. Do you want to continue? [Y/n]:y
{"log.level":"info","@timestamp":"2026-05-09T03:42:53.746+0200","log.origin":{"function":"github.com/elastic/elastic-agent/internal/p
kg/agent/cmd.(*enrollCmd).prepareFleetTLS","file.name":"cmd/enroll_cmd.go","file.line":464},"message":"Generating self-signed certifi
cate for Fleet Server","ecs.version":"1.6.0"}
{"log.level":"info","@timestamp":"2026-05-09T03:42:54.302+0200","log.origin":{"function":"github.com/elastic/elastic-agent/internal/p
kg/agent/cmd.(*enrollCmd).daemonReloadWithBackoff","file.name":"cmd/enroll_cmd.go","file.line":508},"message":"Restarting agent daemo
n, attempt 0","ecs.version":"1.6.0"}
{"log.level":"info","@timestamp":"2026-05-09T03:42:56.314+0200","log.origin":{"function":"github.com/elastic/elastic-agent/internal/p
kg/agent/cmd.waitForFleetServer.func1","file.name":"cmd/enroll_cmd.go","file.line":862},"message":"Fleet Server - Starting","ecs.vers
ion":"1.6.0"}
{"log.level":"info","@timestamp":"2026-05-09T03:43:00.335+0200","log.origin":{"function":"github.com/elastic/elastic-agent/internal/p
```

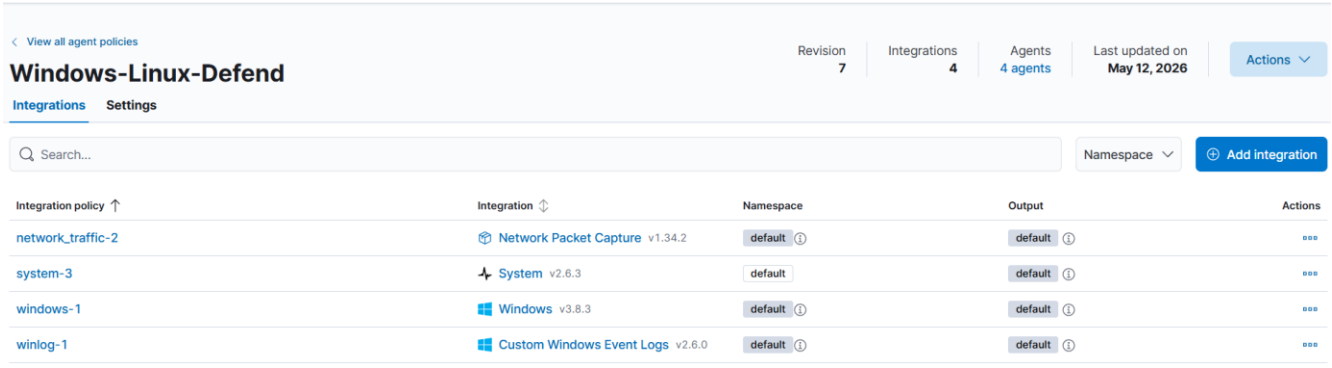
8. Install Elastic Defend Agent on Endpoints

Step 1: Create Elastic Defend Policy

5. Go to **Kibana**
6. Navigate to:

Security → Manage → Policies

7. Click **Create Policy**
8. Enable:
   - Elastic Defend Integration
9. Select protection mode:
   - Prevent / Detect (based on lab setup)

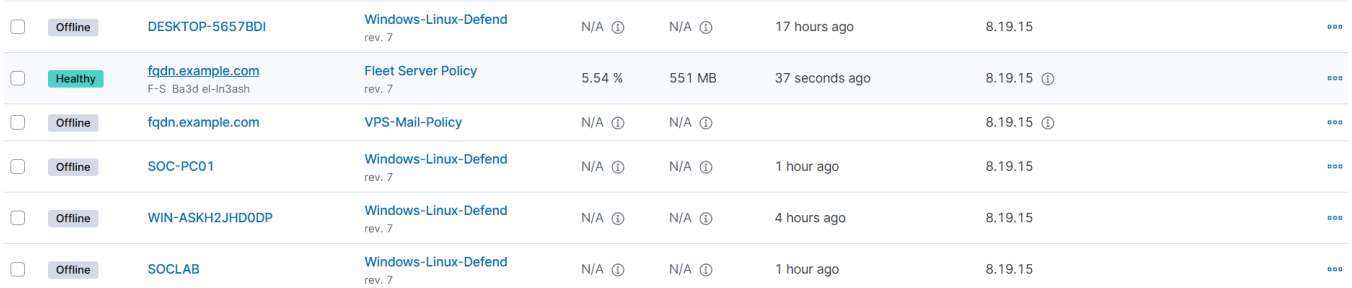


Step 2: Add Agent in Fleet

1. Go to:

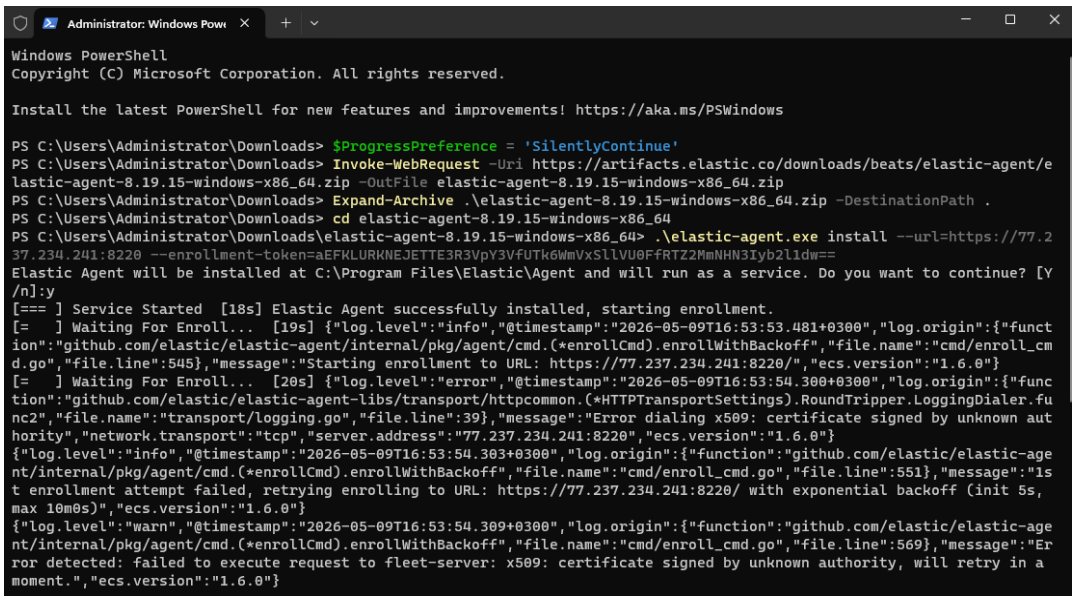
Management → Fleet → Agents

2. Click **Add Agent**
3. Select the created policy (Elastic Defend Policy)
4. Choose platform:
   - Windows / Linux / Ubuntu
5. Copy the installation command provided by Kibana

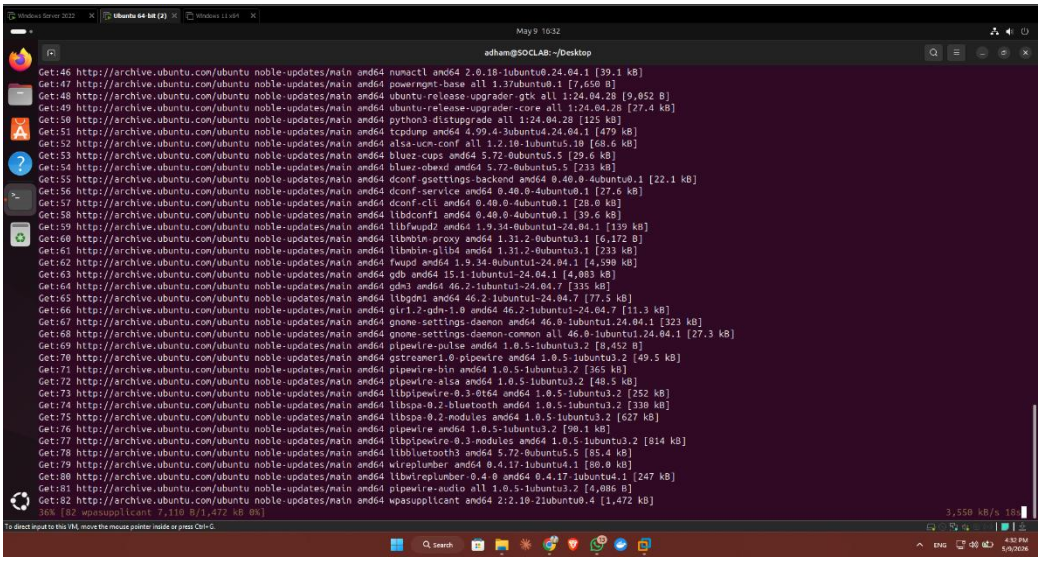


Step 3: Install Elastic Defend Agent on Endpoints (Windows – Linux)

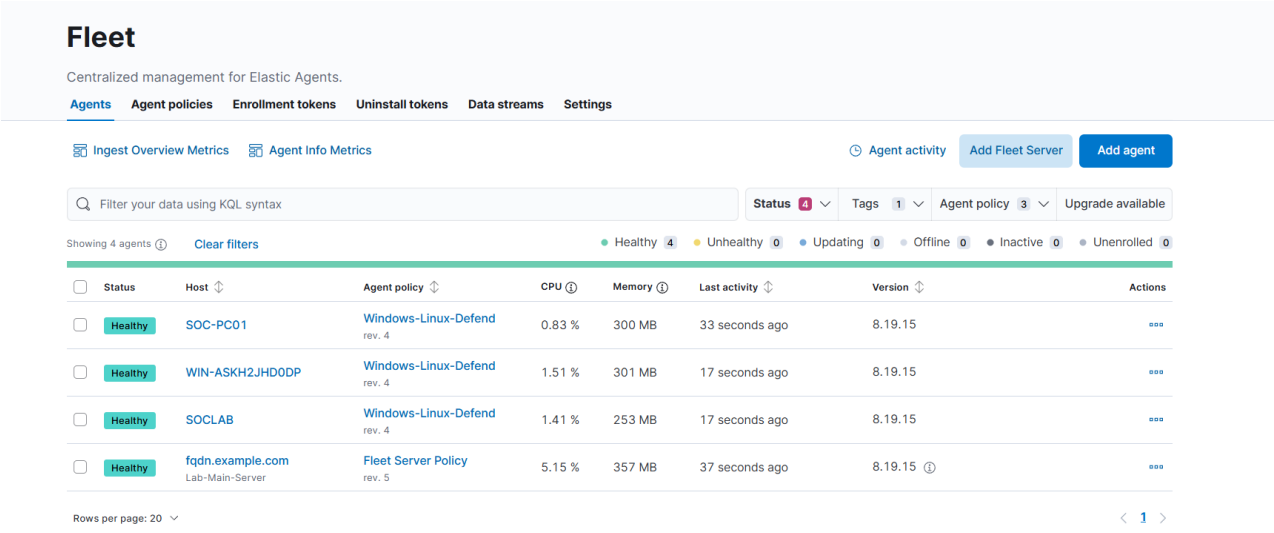
```
$ProgressPreference = 'SilentlyContinue'
Invoke-WebRequest -Uri https://artifacts.elastic.co/downloads/beats/elastic-agent/elastic-agent-8.19.15-windows-x86_64.zip -OutFile elastic-agent-8.19.15-windows-x86_64.zip
Expand-Archive .\elastic-agent-8.19.15-windows-x86_64.zip -DestinationPath .
cd elastic-agent-8.19.15-windows-x86_64
.\elastic-agent.exe install --url=https://77.237.234.241:8220 --enrollment-token=aEFKLURKNEJETTE3R3VpY3VfUTk6WmVxSllVU0FfRTZ2MmNHN3Iyb2l1dw==
```




```
curl -L -O https://artifacts.elastic.co/downloads/beats/elastic-agent/elastic-agent-8.19.15-linux-x86_64.tar.gz
tar xzvf elastic-agent-8.19.15-linux-x86_64.tar.gz
cd elastic-agent-8.19.15-linux-x86_64
sudo ./elastic-agent install --url=https://77.237.234.241:8220 --enrollment-token=aEFKLURKNEJETTE3R3VpY3VfUTk6WmVxSllVU0FfRTZ2MmNHN3Iyb2l1dw==
```



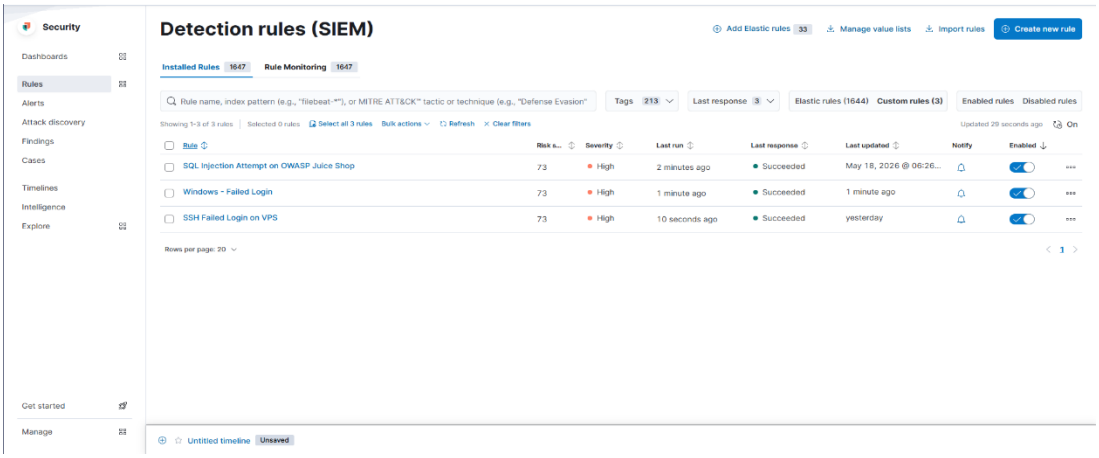
Step 4: Verify Healthy Enrollment



9.Create Custom rules

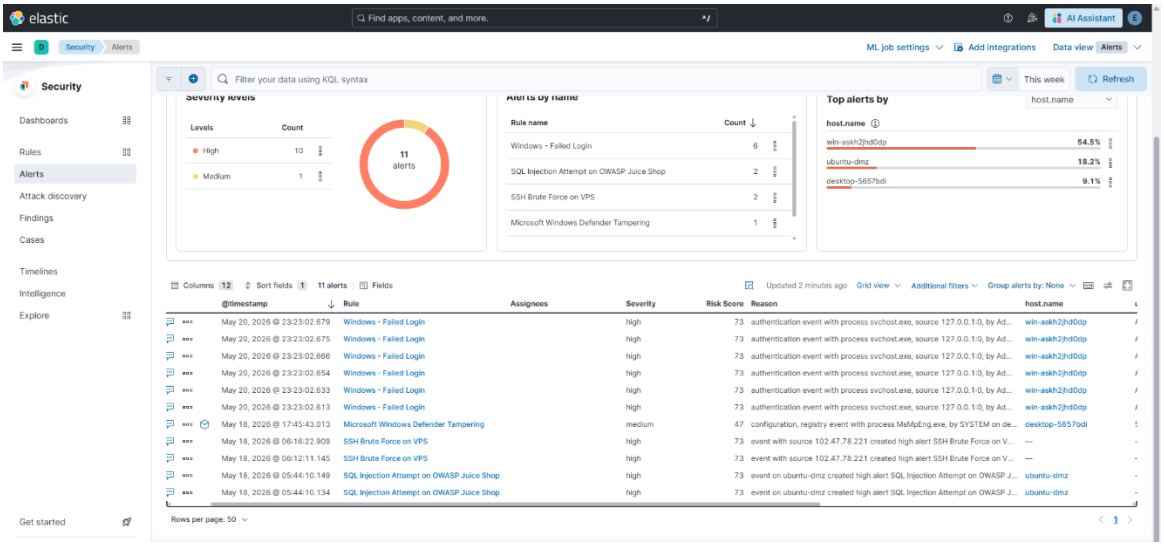
As part of the Mini SOC implementation, several custom detection rules were created in Kibana to identify common attack patterns and simulate real-world threat scenarios.

- **Windows Failed Login (Event ID 4625):**
  Detects authentication failures to identify unauthorized access attempts and possible brute-force attacks.
- **SSH Brute Force (Linux):**
  Monitors repeated failed SSH login attempts to detect brute-force activity targeting Linux systems.
- **SQL Injection Attempts:**
  Identifies suspicious web requests containing malicious SQL patterns such as UNION, SELECT, and DROP



10.Alerts & Incident Notification

The Mini SOC system generates real-time alerts based on the defined detection rules in Kibana. These alerts provide immediate visibility into suspicious activities such as failed logins, brute-force attempts, and potential web attacks.
Each alert includes key details such as the affected host, user account, source IP, and event context, enabling efficient triage and investigation. Alerts are prioritized based on severity levels (Low, Medium, High) to help analysts focus on critical threats.
This alerting mechanism ensures rapid detection, streamlined incident response, and improved overall security monitoring within the SOC environment.



11. Conclusion

- **Infrastructure Summary & Lessons Learned**

The Integrated Mini-SOC Ecosystem successfully demonstrated the design, deployment, and operationalization of a production-grade security monitoring environment built entirely on open-source and cloud-based technologies. From an infrastructure standpoint, the project validated a hybrid architecture model — combining cloud VPS hosting with on-premises virtualized endpoints — as a cost-effective and scalable foundation for a functional SOC.

- **Architecture Validation**

The decision to host the SIEM core (Elasticsearch 8.x + Kibana + Fleet Server) on a Contabo VPS running Ubuntu 22.04 proved to be the right call for ensuring 24/7 log availability and centralized policy management. Decoupling the data collection layer (endpoints) from the analytics layer (cloud)

introduced realistic network segmentation and accurately mirrors enterprise-grade SOC architectures where agents report to a centralized platform over the internet.

The on-premises cluster — consisting of a Windows Server 2022 Domain Controller, a Windows 11 Pro domain-joined workstation, and an Ubuntu GUI machine — provided a diverse, multi-OS log source environment. This setup generated meaningful telemetry covering Windows Event Logs, Active Directory authentication events, and Linux syslog data, all forwarded to the ELK stack via Elastic Agent with Elastic Defend for kernel-level EDR visibility.

- **Deployment Challenges & Engineering Decisions**

The project encountered real-world infrastructure challenges that added significant learning value. The JVM heap tuning (setting -Xms4g / -Xmx4g on a 12GB RAM VPS) was a critical optimization step to prevent Elasticsearch from consuming all available memory and causing service instability. This is a common pain point in production Elastic deployments and was handled correctly.

The Access Denied exception during Fleet Server enrollment — caused by incorrect file ownership on /etc/elasticsearch/service_tokens — is a classic Linux permission hardening issue. Resolving it through chown and chmod 660 reflects solid sysadmin fundamentals applied in a security context.

SSL/TLS certificate generation using elasticsearch-certutil for securing agent-to-Fleet communication further strengthened the infrastructure's security posture and ensured encrypted log transport across all nodes.

- **Key Outcomes**

The final infrastructure achieved the following operational capabilities:

  - Centralized ingestion and indexing of multi-source security logs in near real-time
  - Agent-based endpoint coverage across Windows and Linux platforms via Elastic Defend (EDR)
  - Fleet-managed policy enforcement across all enrolled agents from a single control plane
  - Secure communications enforced via self-signed TLS certificates throughout the stack

- **Final Assessment**

This project delivered a technically sound, end-to-end SOC infrastructure that bridges the gap between theoretical SOC architecture and hands-on deployment. Every component — from OS-level configuration to certificate management and agent enrollment — was implemented with security and operational stability in mind. The experience gained directly mirrors the challenges faced by infrastructure and SOC teams in enterprise environments, making it a strong foundation for real-world blue team operations.